STUDY GUIDE

Practice Workbook

242 exercises grouped by module — no solutions

Solutions are in the separate Solutions Manual so you are not tempted to peek.

Generated 10 October 2026 from the same lesson data that powers the website.
Every Java program in this guide was compiled with --release 17 and run against its expected output.

Part 0 · Programming & Java Setup

0.1 How Java Works: Source, Bytecode, JVM, JRE, JDK

Exercise 1 · Print your own message — *Beginner*

Write a program that prints exactly: `Write once, run anywhere`

Example input	Expected output
(no input)	Write once, run anywhere

Hint 1: System.out.println("...");

0.2 The Java Landscape: History, Features, Editions, JVM Memory

Exercise 1 · Powers of two in memory units — *Beginner*

Read a number of megabytes M and print how many bytes that is (1 MB = 1024 × 1024 bytes). Use long.

- 0 ≤ M ≤ 8 000 000 000

Example input	Expected output
256	268435456

Hint 1: long bytes = (long) M * 1024 * 1024;

0.3 Installing Java on macOS and Choosing an Editor

Exercise 1 · Compile and run it yourself — *Beginner*

In the playground or your terminal run the verification program. Then write a program that prints the two lines `JDK ready` and `Let us begin`.

Example input	Expected output
(no input)	JDK ready ⏎ Let us begin

Hint 1: Two System.out.println statements.

0.4 Your First Program: class, main, statements, comments

Exercise 1 · Print a box — *Beginner*

Print exactly these three lines: `` `+--+ |Hi| +--+` ``

Example input	Expected output
(no input)	+--+ ⏎ |Hi| ⏎ +--+

Hint 1: Three println statements.

Exercise 2 · Echo a name — *Beginner*

Read a name (one line) and print `Hello, <name>!`

Example input	Expected output
Asha	Hello, Asha!

Hint 1: Scanner.nextLine() and string concatenation.

Exercise 3 · Sum two numbers — *Intermediate*

Read two integers on one line and print their sum.

Example input	Expected output
7 5	12

Hint 1: sc.nextInt() twice.

0.5 Compile-time, Runtime and Logic Errors

Exercise 1 · Correct average — *Beginner*

Read two integers a and b and print their average as a decimal with exactly one digit after the point (e.g. 2.5).

Example input	Expected output
2 3	2.5

Hint 1: Divide by 2.0, not 2. **Hint 2:** System.out.printf("%.1f%n", value);

Exercise 2 · Safe division — *Intermediate*

Read two integers a and b. Print a / b as integer division, or the text `cannot divide by zero` if b is 0 (without crashing).

Example input	Expected output
7 2	3
5 0	cannot divide by zero

Hint 1: Check b first, or catch ArithmeticException.

0.6 What Is an Algorithm?

Exercise 1 · Find the smallest — *Beginner*

Read n and n integers. Print the smallest.

Example input	Expected output
4 ⏎ 5 2 8 1	1

Hint 1: Mirror the "largest" algorithm with < instead of >.

Exercise 2 · Sum of the numbers — *Beginner*

Read n and n integers. Print their sum.

Example input	Expected output
4 ⏎ 5 2 8 1	16

Hint 1: An accumulator variable that starts at 0.

Exercise 3 · Is it sorted? — *Intermediate*

Read n and n integers. Print YES if they are in non-decreasing order, otherwise NO.

Example input	Expected output
4 ⏎ 1 2 2 3	YES

Hint 1: Compare each number with the previous one.

Part 1 · Java Syntax & Fundamentals

1.1 Variables, Constants and Primitive Types

Exercise 1 · Swap two numbers — *Beginner*

Read two integers a and b and print them swapped: `b a`.

Example input	Expected output
3 8	8 3

Hint 1: You need a third variable to hold one value temporarily.

Exercise 2 · Rectangle — *Beginner*

Read the width and height (decimals). Print `area=…` and `perimeter=…` on two lines using default double printing.

Example input	Expected output
2.5 4	area=10.0 ⏎ perimeter=13.0

Hint 1: double area = w * h;

Exercise 3 · Seconds to h:m:s — *Intermediate*

Read a number of seconds and print it as `hours:minutes:seconds` (no leading zeros).

Example input	Expected output
3725	1:2:5

Hint 1: hours = total / 3600; minutes = (total % 3600) / 60; seconds = total % 60.

1.2 Type Conversion, Casting and Overflow

Exercise 1 · Big product — *Beginner*

Read two ints a, b (up to 10^9 each) and print a × b exactly.

Example input	Expected output
100000 100000	10000000000

Hint 1: Cast one operand to long before multiplying.

Exercise 2 · Shift a letter — *Intermediate*

Read an uppercase letter and k ($0 \le k \le 1000$). Print the letter k places further in the alphabet, wrapping Z to A.

Example input	Expected output
Y 3	B

Hint 1: ('A' + (c - 'A' + k) % 26) then cast to char.

Exercise 3 · Will it overflow? — *Intermediate*

Read two long values a and b. Print OK if a + b fits in an int, otherwise OVERFLOW.

Example input	Expected output
2000000000 2000000000	OVERFLOW

Hint 1: Compute the sum as long and compare with Integer.MIN_VALUE / MAX_VALUE.

1.3 Arithmetic, Assignment and Increment Operators

Exercise 1 · Digit sum — *Beginner*

Read a positive integer and print the sum of its digits.

Example input	Expected output
4821	15

Hint 1: while (n > 0) { sum += n % 10; n /= 10; }

Exercise 2 · Reverse the digits — *Beginner*

Read a positive integer and print it reversed (trailing zeros of the input vanish: 1200 → 21).

Example input	Expected output
1200	21

Hint 1: reversed = reversed * 10 + n % 10

Exercise 3 · Days to weeks — *Beginner*

Read a number of days and print `W weeks D days`.

Example input	Expected output
30	4 weeks 2 days

Hint 1: days / 7 and days % 7

1.4 Relational, Logical and Ternary Operators; Precedence

Exercise 1 · Leap year — *Beginner*

Read a year. Print `leap` if it is a leap year (divisible by 4, except centuries unless divisible by 400), otherwise `common`.

Example input	Expected output
2000	leap

Hint 1: (y % 4 == 0 && y % 100 != 0) || y % 400 == 0

Exercise 2 · Maximum of three — *Beginner*

Read three integers and print the largest using only ternary operators.

Example input	Expected output
4 9 7	9

Hint 1: Nest a ternary inside a ternary.

Exercise 3 · In range? — *Intermediate*

Read x, lo and hi. Print IN if lo ≤ x ≤ hi, else OUT.

Example input	Expected output
5 1 10	IN

Hint 1: Two comparisons joined by &&.

1.5 Bitwise Operators and Shifts

Exercise 1 · Power of two? — *Beginner*

Read a positive integer (up to 10^{18}). Print YES if it is a power of two, otherwise NO.

Example input	Expected output
64	YES

Hint 1: n & (n − 1) clears the lowest set bit; a power of two has only one.

Exercise 2 · Count the 1 bits — *Intermediate*

Read a non-negative int and print how many 1 bits its binary form has (without Integer.bitCount).

Example input	Expected output
255	8

Hint 1: n &= n − 1 removes one set bit each time.

Exercise 3 · Set, clear or toggle — *Intermediate*

Read n, a bit index i (0-based) and an operation word `set`, `clear` or `toggle`. Print the resulting integer.

Example input	Expected output
5 1 set	7

Hint 1: mask = 1 << i; then |, & ~mask, or ^.

1.6 Output: print, println, printf and String Concatenation

Exercise 1 · Times table — *Beginner*

Read n and print `n x i = n·i` for i = 1..5, one line each.

Example input	Expected output
7	7 x 1 = 7 ⏎ 7 x 2 = 14 ⏎ 7 x 3 = 21 ⏎ 7 x 4 = 28 ⏎ 7 x 5 = 35

Hint 1: printf("%d x %d = %d%n", ...)

Exercise 2 · Invoice total — *Intermediate*

Read a unit price (decimal) and a quantity. Print `Total: X` with thousands separators and exactly two decimals.

Example input	Expected output
1999.5 3	Total: 5,998.50

Hint 1: %,.2f

Exercise 3 · Predict, then run — *Beginner*

The program prints the value of `a + b + "=" + a + b` for a = 2, b = 3. Run it and write what it prints (this exercise checks that you reproduce the output exactly).

Example input	Expected output
(no input)	5=23

Hint 1: Left to right.

1.7 Input: Scanner, BufferedReader and Command-Line Arguments

Exercise 1 · Average — *Beginner*

Read n and then n decimals. Print their average with two decimals.

Example input	Expected output
3 ⏎ 1.5 2.5 3.5	2.50

Hint 1: printf("%.2f%n", sum / n)

Exercise 2 · Sum until the end — *Intermediate*

The input has any number of integers over several lines. Print their sum.

Example input	Expected output
4 5 6 ⏎ 7	22

Hint 1: while (sc.hasNextLong())

Exercise 3 · Age then name — *Intermediate*

The first line has an age, the second a full name (with spaces). Print `<name> is <age>`.

Example input	Expected output
20 ⏎ Asha Rao	Asha Rao is 20

Hint 1: Consume the newline after nextInt().

1.8 Decision Making: if, else, switch

Exercise 1 · Grade — *Beginner*

Read marks and print A (≥ 90), B (≥ 75), C (≥ 50) or F.

Example input	Expected output
82	B

Hint 1: Highest range first.

Exercise 2 · Calculator — *Intermediate*

Read `a op b` with op one of + - * /. Print the result of integer arithmetic; for / by zero print `error`; for any other operator print `unknown operator`.

Example input	Expected output
12 / 4	3

Hint 1: switch on the char.

Exercise 3 · Triangle type — *Intermediate*

Read three side lengths. Print `not a triangle`, `equilateral`, `isosceles` or `scalene`.

Example input	Expected output
5 5 8	isosceles

Hint 1: The triangle inequality: every side must be shorter than the sum of the other two.

1.9 Loops: for, while, do-while, break and continue

Exercise 1 · Factorial — *Beginner*

Read n (0 ≤ n ≤ 20) and print n! (use long).

Example input	Expected output
10	3628800

Hint 1: Start with 1 and multiply by 2, 3, …, n.

Exercise 2 · Pyramid — *Intermediate*

Read n and print a pyramid of n rows of `*`, row i has 2i−1 stars, centred with leading spaces.

Example input	Expected output
3	* ⏎ * ⏎ ***

Hint 1: Row r: n − r spaces then 2r − 1 stars.

Exercise 3 · Collatz steps — *Intermediate*

Start from n. If n is even divide it by 2, otherwise compute 3n + 1. Count the steps until n becomes 1.

Example input	Expected output
6	8

Hint 1: Use long; a while loop with a counter.

1.10 Methods: Parameters, Return Values, Scope and Overloading

Exercise 1 · isPrime method — *Beginner*

Write `static boolean isPrime(int n)` and use it to print `prime` or `not prime` for the input.

Example input	Expected output
97	prime

Hint 1: Test divisors from 2 while d·d ≤ n.

Exercise 2 · Overloaded power — *Intermediate*

Write two methods named `power`: one for (long base, int exp) and one for (double base, int exp). Read base and exp and print power(base, exp) as long, then print power(2.5, 2).

Example input	Expected output
3 4	81 ⏎ 6.25

Hint 1: Multiply in a loop.

Exercise 3 · Digits helpers — *Intermediate*

Using two helper methods print `<number of digits> <sum of digits>` for a non-negative integer (0 has 1 digit).

Example input	Expected output
90210	5 12

Hint 1: A do-while loop counts digits correctly even for 0.

1.11 Tracing and Debugging Programs

Exercise 1 · Trace by hand — *Beginner*

What does the program print? (Trace x = 1; for i = 0..3: x = x·2 + i.) Run it to check.

Example input	Expected output
(no input)	27

Hint 1: Write the trace table.

Exercise 2 · Print the trace — *Intermediate*

Read a start value and a number of steps. Repeat value = value·2 + 1 for that many steps and print `step value` after each.

Example input	Expected output
1 3	1 3 ⏎ 2 7 ⏎ 3 15

Hint 1: A loop with one println inside.

Exercise 3 · Fix the maximum — *Intermediate*

A buggy program started `max = 0` and so failed on all-negative inputs. Write the correct version: read n and n integers; print the largest.

Example input	Expected output
3 ⏎ −4 −2 −9	−2

Hint 1: Start from Integer.MIN_VALUE or the first element.

Part 2 · Core Programming Skills

2.1 One-Dimensional Arrays

Exercise 1 · Print reversed — *Beginner*

Read n and an array; print its elements in reverse order separated by spaces.

Example input	Expected output
5 ⏎ 1 2 3 4 5	5 4 3 2 1

Hint 1: Loop from n − 1 down to 0.

Exercise 2 · Digit frequencies — *Intermediate*

Read n digits (0–9). For every digit that occurs print `digit: count` in increasing digit order.

Example input	Expected output
6 ⏎ 3 1 3 0 9 1	0: 1 ⏎ 1: 2 ⏎ 3: 2 ⏎ 9: 1

Hint 1: A counting array of size 10: freq[d]++.

Exercise 3 · Second largest distinct — *Challenge*

Print the second largest DISTINCT value, or -1 if there is none. One pass, O(1) extra space.

Example input	Expected output
5 ⏎ 4 9 9 2 7	7

Hint 1: Track first and second; update second only when x is strictly between them.

2.2 Two-Dimensional Arrays (Matrices)

Exercise 1 · Sum of a matrix — *Beginner*

Read R, C and the matrix. Print the sum of all elements.

Example input	Expected output
2 3 ⏎ 1 2 3 ⏎ 4 5 6	21

Hint 1: Two nested loops.

Exercise 2 · Transpose — *Intermediate*

Read R, C and the matrix. Print the C × R transpose, one row per line.

Example input	Expected output
2 3 ⏎ 1 2 3 ⏎ 4 5 6	1 4 ⏎ 2 5 ⏎ 3 6

Hint 1: Swap the loop order when printing.

Exercise 3 · Spiral order — *Challenge*

Read n and an n × n matrix. Print its elements in clockwise spiral order starting at the top-left, on one line.

Example input	Expected output
3 ⏎ 1 2 3 ⏎ 4 5 6 ⏎ 7 8 9	1 2 3 6 9 8 7 4 5

Hint 1: Keep four boundaries: top, bottom, left, right and shrink them.

2.3 Strings, StringBuilder and Characters

Exercise 1 · Count vowels — *Beginner*

Read a line and print how many vowels (a, e, i, o, u, case-insensitive) it contains.

Example input	Expected output
`Programming Language`	`7`

Hint 1: "aeiou".indexOf(c) >= 0

Exercise 2 · Anagrams — *Intermediate*

Read two words. Print YES if they are anagrams (same letters, same counts, ignoring case), else NO.

Example input	Expected output
`Listen Silent`	`YES`

Hint 1: A 26-element count array: ++ for the first word, −− for the second.

Exercise 3 · Run-length encoding — *Intermediate*

Read a word and print its run-length encoding: each maximal block of equal characters becomes the character followed by the block length.

Example input	Expected output
`aaabccdddd`	`a3b1c2d4`

Hint 1: A while loop with an inner scan for the end of the run.

2.4 Number Systems, Binary and Two's Complement

Exercise 1 · To binary — *Beginner*

Read a non-negative integer and print its binary representation (without Integer.toBinaryString).

Example input	Expected output
`156`	`10011100`

Hint 1: Repeated division by 2; reverse the remainders; handle 0.

Exercise 2 · From any base — *Intermediate*

Read a number written in base b (2 ≤ b ≤ 16) and the base. Print its decimal value.

Example input	Expected output
`9C 16`	`156`

Hint 1: value = value * base + digit

Exercise 3 · Add two binary strings — *Challenge*

Read two binary numbers as strings (up to 1000 digits) and print their sum in binary, without converting to int.

Example input	Expected output
`1010 1110`	`11000`

Hint 1: Add digit by digit from the right with a carry.

2.5 Divisibility, GCD/LCM, Primes and Modular Arithmetic

Exercise 1 · gcd and lcm — *Beginner*

Read a and b (up to 10^9). Print gcd and lcm separated by a space.

Example input	Expected output
12 18	6 36

Hint 1: lcm = a / gcd * b with long.

Exercise 2 · All divisors — *Intermediate*

Read n (up to 10^{12}) and print all divisors in increasing order.

Example input	Expected output
36	1 2 3 4 6 9 12 18 36

Hint 1: For each d ≤ √n add both d and n/d to a sorted set.

Exercise 3 · Modular power — *Challenge*

Read a, b, m (a, m ≤ 10^9, b ≤ 10^{18}). Print a^b mod m.

Example input	Expected output
3 200 13	9

Hint 1: Repeated squaring; use long and reduce after every multiplication.

2.6 Recursion: Base Case, Recursive Case and the Call Stack

Exercise 1 · Recursive digit sum — *Beginner*

Read n ≥ 0 and print the sum of its digits using recursion (no loops).

Example input	Expected output
4821	15

Hint 1: sumDigits(n) = n % 10 + sumDigits(n / 10); base case n < 10.

Exercise 2 · Towers of Hanoi — *Intermediate*

Move n disks from rod A to rod C using B. Print each move as `Move disk k from X to Y`, then `total moves: M`.

- 1 ≤ n ≤ 10

Example input	Expected output
2	Move disk 1 from A to B ⏎ Move disk 2 from A to C ⏎ Move disk 1 from B to C ⏎ total moves: 3

Hint 1: Move n−1 disks to the spare rod, move the biggest, move n−1 on top of it.

Exercise 3 · All subsets — *Challenge*

Read n (1 ≤ n ≤ 10). Print all subsets of {1..n} in the order produced by "skip i" before "take i" (see example).

Example input	Expected output
2	[] ⏎ [2] ⏎ [1] ⏎ [1, 2]

Hint 1: subsets(i): skip i then take i, undo after taking.

2.7 References, Pass-by-Value and Memory

Exercise 1 · Swap inside an array — *Beginner*

Read n, the array and two indices i and j. Print the array after swapping a[i] and a[j] using a method `swap(int[] a, int i, int j)`.

Example input	Expected output
4 ⏎ 1 2 3 4 ⏎ 0 3	[4, 2, 3, 1]

Hint 1: The method changes the shared array, so it works.

Exercise 2 · Return a new array — *Intermediate*

Read n and an array. Print the original and a NEW array with every element doubled, as `[orig] [doubled]`.

Example input	Expected output
3 ⏎ 1 2 3	[1, 2, 3] [2, 4, 6]

Hint 1: Allocate a second array; do not modify the first.

Exercise 3 · Alias or copy? — *Beginner*

Read mode 1 or 2. Mode 1: b = a; mode 2: b = a.clone(). Then set b[0] = 99 and print `same` if a[0] also became 99, else `different`.

Example input	Expected output
1	same

Hint 1: clone() creates a new array.

2.8 Boundary Conditions, Validation and Common Mistakes

Exercise 1 · Sum the valid numbers — *Beginner*

Read whitespace-separated tokens. Print `sum bad` where sum is the total of the tokens that are valid ints and bad is how many are not.

Example input	Expected output
5 x 7 3.5 -2	10 2

Hint 1: try { Integer.parseInt } catch (NumberFormatException e)

Exercise 2 · Maximum or empty — *Beginner*

Read n, then n integers. Print the maximum, or `empty` when n = 0.

Example input	Expected output
`0`	`empty`

Hint 1: Handle n = 0 first.

Exercise 3 · Safe element access — *Intermediate*

Read n, an array and an index i. Print a[i] if 0 ≤ i < n, else `out of range`.

Example input	Expected output
`3 ⏎ 10 20 30 ⏎ 3`	`out of range`

Hint 1: Check both ends of the range.

Part 3 · Object-Oriented Programming

3.1 Classes, Objects, Fields, Constructors and this

Exercise 1 · Rectangle class — *Beginner*

Create a class Rectangle with width and height, and methods area() and perimeter(). Read width and height (decimals) and print `area perimeter`.

Example input	Expected output
2.5 4	10.0 13.0

Hint 1: A constructor that stores both fields.

Exercise 2 · Counter — *Intermediate*

Create a Counter with increment(), add(k) and get(). Read n operations: `inc` or `add k`. Print the final value.

Example input	Expected output
4 ⏎ inc ⏎ add 5 ⏎ inc ⏎ add 10	17

Hint 1: A private int field.

Exercise 3 · Distance between points — *Intermediate*

Create a Point class with a method distanceTo(Point). Read x1 y1 x2 y2 and print the distance with three decimals.

Example input	Expected output
0 0 3 4	5.000

Hint 1: Math.sqrt(dx*dx* + *dy*dy)

3.2 Encapsulation and Access Modifiers

Exercise 1 · Validated temperature — *Beginner*

Create a Temperature class whose setCelsius rejects values below −273.15 by throwing IllegalArgumentException. Read a Celsius value; print the Fahrenheit value with one decimal, or `invalid: below absolute zero`.

Example input	Expected output
36.6	97.9

Hint 1: catch the exception in main.

Exercise 2 · Bounded stack — *Intermediate*

Implement an int stack of capacity 3 with push (returns false if full) and pop. Read n commands (`push x` or `pop`); print `ok`/`full` for pushes and the popped value or `empty` for pops.

Example input	Expected output
5 ⏎ push 1 ⏎ push 2 ⏎ pop ⏎ pop ⏎ pop	ok ok 2 1 empty

Hint 1: A private array and a private size.

Exercise 3 · Access levels — *Beginner*

Run the program: it prints the values of four fields declared with private, package-private, protected and public inside a class. (The output `42 7 3 1` shows they are all reachable from inside the same top-level class.)

Example input	Expected output
(no input)	42 7 3 1

Hint 1: Read the lesson table: same-class access is always allowed.

3.3 Inheritance, Overriding and Polymorphism

Exercise 1 · Animal sounds — *Beginner*

Create Animal (sound "..."), Dog ("Woof") and Cat ("Meow"). Read n kinds (`dog`, `cat` or anything else = plain animal) and print each sound separated by spaces.

Example input	Expected output
3 ⏎ dog ⏎ cat ⏎ fish	Woof Meow ...

Hint 1: Store each as an Animal reference and call sound().

Exercise 2 · Payroll — *Intermediate*

Employee pays base; Manager pays 1.5 × base. Read n lines `role name base` (role manager or staff) and print the total pay with two decimals.

Example input	Expected output
2 ⏎ manager Ann 1000 ⏎ staff Bob 800	2300.00

Hint 1: Override pay() in Manager.

Exercise 3 · Predict the order — *Beginner*

Run the program and write down the output: the constructors of a parent and child class, and a call through a parent-typed variable.

Example input	Expected output
(no input)	A() ⏎ B() ⏎ B

Hint 1: Parent constructor first; the object is a B.

3.4 Abstract Classes and Interfaces

Exercise 1 · Total area — *Beginner*

Define an interface Shape with area(); implement Circle and Square. Read n lines `circle r` or `square s` and print the total area with two decimals.

Example input	Expected output
3 ⏎ circle 1 ⏎ square 2 ⏎ circle 2	19.71

Hint 1: A list/array of Shape.

Exercise 2 · Abstract Vehicle — *Intermediate*

Create an abstract class Vehicle with an abstract wheels() and a concrete describe() that prints `<ClassName> has <n> wheels`. Read `bike` or `car` and print the description.

Example input	Expected output
bike	Bike has 2 wheels

Hint 1: getClass().getSimpleName()

Exercise 3 · Comparable versions — *Intermediate*

Create class Version implementing Comparable<Version> for strings like `1.10`. Read n versions and print them sorted by major then minor numerically.

Example input	Expected output
4 ⏎ 1.10 ⏎ 1.2 ⏎ 2.0 ⏎ 1.9	1.2 1.9 1.10 2.0

Hint 1: Integer.compare on major, then minor.

3.5 static, final and Initialisation Order

Exercise 1 · ID generator — *Beginner*

Create a class with a static counter and a static method nextId(). Read n and print n generated IDs starting at 1, space separated.

Example input	Expected output
4	1 2 3 4

Hint 1: private static int next = 1; return next++;

Exercise 2 · A constant — *Beginner*

Define `static final double PI_APPROX = 3.14159` and a method circleArea(r). Read r and print the area with three decimals.

Example input	Expected output
2	12.566

Hint 1: printf("%.3f%n", ...)

Exercise 3 · Initialisation order — *Intermediate*

Run the program that creates two objects of a class with a static block, an instance block and a constructor; write down the order of the five lines.

Example input	Expected output
(no input)	static block ⏎ instance block ⏎ constructor ⏎ instance block ⏎ constructor

Hint 1: Static block once; instance block + constructor for every object.

3.6 Packages and Imports

Exercise 1 · Static import — *Beginner*

Use a static import of Math.sqrt and print the square root of the input number.

Example input	Expected output
16	4.0

Hint 1: import static java.lang.Math.sqrt;

Exercise 2 · Sort with Arrays — *Beginner*

Read n and an array. Import java.util.Arrays and print the sorted array as `[a, b, c]`.

Example input	Expected output
4 ⏎ 9 3 7 1	[1, 3, 7, 9]

Hint 1: Arrays.sort then Arrays.toString

Exercise 3 · Same name, two packages — *Intermediate*

Create a java.util.Date and a java.sql.Date using fully qualified names and print their class names (one per line).

Example input	Expected output
(no input)	java.util.Date ⏎ java.sql.Date

Hint 1: No imports needed.

3.7 Exceptions: try, catch, finally, throw and throws

Exercise 1 · Safe division — *Beginner*

Read two tokens. Print a / b as integers, `bad number` if either is not an integer, or `division by zero`.

Example input	Expected output
12 4	3

Hint 1: Two catch clauses.

Exercise 2 · Custom exception — *Intermediate*

Define NegativeAgeException (unchecked). Read n ages; print `ok <age>` for valid ones, or the exception message `age cannot be negative: <age>`.

Example input	Expected output
3 ⏎ 20 ⏎ −5 ⏎ 0	ok 20 ⏎ age cannot be negative: −5 ⏎ ok 0

Hint 1: extends RuntimeException; call super(message).

Exercise 3 · try / finally order — *Intermediate*

Run the program: a method prints in try, returns 1, and prints in finally. Write the output order.

Example input	Expected output
(no input)	try ⏎ finally ⏎ 1

Hint 1: finally runs before the value is delivered to the caller.

3.8 Generics and Lists: ArrayList and LinkedList

Exercise 1 · Remove duplicates — *Beginner*

Read n integers and print them with duplicates removed, keeping the first occurrence of each.

Example input	Expected output
6 ⏎ 3 1 3 2 1 4	3 1 2 4

Hint 1: Add to a result list only if !contains.

Exercise 2 · Remove a value — *Intermediate*

Read n integers and a target. Print the list (as `[a, b]`) with every occurrence of the target removed — without ConcurrentModificationException.

Example input	Expected output
6 ⏎ 3 1 3 2 1 4 ⏎ 3	[1, 2, 1, 4]

Hint 1: list.removeIf

Exercise 3 · Generic maximum — *Challenge*

Write `static <T extends Comparable<T>> T largest(List<T> list)` and use it on n words; print the largest alphabetically.

Example input	Expected output
4 ⏎ pear apple zebra fig	zebra

Hint 1: `T extends Comparable<T>` lets you call compareTo.

3.9 Sets and Maps: HashSet, TreeSet, HashMap, TreeMap

Exercise 1 · Distinct count — *Beginner*

Read n integers and print how many distinct values there are.

Example input	Expected output
6 ⏎ 3 1 3 2 1 4	4

Hint 1: A HashSet and its size().

Exercise 2 · Word frequency — *Intermediate*

Read words until the end of input (case-insensitive). Print each distinct word and its count, one `word count` per line, alphabetically.

Example input	Expected output
the cat and The hat	and 1 ⏎ cat 1 ⏎ hat 1 ⏎ the 2

Hint 1: TreeMap<String,Integer> + merge.

Exercise 3 · Two-sum — *Challenge*

Read n, target and an array. Print the indices `i j` (i < j) of the pair with the smallest j such that a[i] + a[j] = target, or `none`. Use O(n).

Example input	Expected output
4 18 ⏎ 2 7 11 15	1 2

Hint 1: For each j look up target − a[j] among the earlier elements.

3.10 Queues, Deques, Stacks and PriorityQueue

Exercise 1 · Balanced brackets — *Beginner*

Read a string of brackets ()[]{}. Print YES if balanced, else NO.

Example input	Expected output
{[()]}	YES

Hint 1: Push openers; on a closer pop and compare.

Exercise 2 · Josephus with a queue — *Intermediate*

Read n and k. People 1..n stand in a queue; repeatedly move k−1 people to the back and remove the next. Print the removal order.

Example input	Expected output
5 2	2 4 1 5 3

Hint 1: q.add(q.poll()) k−1 times, then poll.

Exercise 3 · k-th largest — *Challenge*

Read n, k and an array. Print the k-th largest element using a min-heap of size k (O(n log k)).

Example input	Expected output
6 2 ⏎ 3 2 1 5 6 4	5

Hint 1: Add each element; if size > k, poll the minimum.

3.11 Sorting with Comparable and Comparator

Exercise 1 · Descending — *Beginner*

Read n integers and print them in descending order.

Example input	Expected output
5 ⏎ 3 9 1 7 5	9 7 5 3 1

Hint 1: Collections.reverseOrder()

Exercise 2 · By length then alphabet — *Intermediate*

Read n words and print them sorted by length, ties alphabetically.

Example input	Expected output
5 ⏎ pear fig apple kiwi plum	fig kiwi pear plum apple

Hint 1: comparingInt(String::length).thenComparing(...)

Exercise 3 · Merge intervals — *Challenge*

Read n intervals `start end`. Merge overlapping ones (touching counts) and print them as `[s,e]` separated by spaces in increasing order.

Example input	Expected output
`4 ⏎ 1 3 ⏎ 8 10 ⏎ 2 6 ⏎ 15 18`	`[1,6] [8,10] [15,18]`

Hint 1: Sort by start, then sweep keeping the current merged interval.

3.12 equals(), hashCode() and Immutability

Exercise 1 · Distinct people — *Intermediate*

Create a Person(name, age) class with equals/hashCode. Read n lines `name age` and print the number of distinct persons.

Example input	Expected output
`4 ⏎ Asha 20 ⏎ Ravi 21 ⏎ Asha 20 ⏎ Asha 22`	`3`

Hint 1: Put Persons in a HashSet.

Exercise 2 · Immutable money — *Intermediate*

Run the Money class whose plus() returns a new object; print the amounts of a and b = a.plus(175¢) for a = 250¢.

Example input	Expected output
`(no input)`	`250 425`

Hint 1: The original object is unchanged.

Exercise 3 · Defensive copying — *Challenge*

A Team stores a copy of the member list given to its constructor and returns an unmodifiable view. Run it: the source list is changed afterwards (the team must not change) and a caller tries to add to the returned list.

Example input	Expected output
`(no input)`	`[Asha, Ravi] ⏎ read-only`

Hint 1: new ArrayList<>(members) in; Collections.unmodifiableList out.

Part 4 · Data Structures & Algorithm Foundations

4.1 Data Structures Overview: Arrays, Linked Lists, Stacks, Queues, Hash Tables

Exercise 1 · Reverse a linked list — *Beginner*

Read n and n numbers into a singly linked list (your own Node class); reverse it and print it.

Example input	Expected output
`5 ⏎ 1 2 3 4 5`	`5 4 3 2 1`

Hint 1: Use three references: prev, cur, next.

Exercise 2 · Palindromic list — *Beginner*

Read n numbers; print YES if the sequence reads the same forwards and backwards.

Example input	Expected output
`5 ⏎ 1 2 3 2 1`	YES

Hint 1: Two indices moving inwards.

Exercise 3 · Circular queue — *Intermediate*

Implement a queue of capacity c on an array. Read c and n commands `enq x` / `deq`; print `ok`/`full` for enq and the removed value or `empty` for deq.

Example input	Expected output
`2 6 ⏎ enq 1 ⏎ enq 2 ⏎ enq 3 ⏎ deq ⏎ enq 3 ⏎ deq`	`ok ok full 1 ok 2`

Hint 1: head and size; index (head + size) % c.

4.2 Trees, Graphs and Heaps

Exercise 1 · Height of a BST — *Beginner*

Read n distinct keys, insert them into a BST in the given order, and print its height (number of nodes on the longest root-to-leaf path).

Example input	Expected output
`7 ⏎ 50 30 70 20 40 60 80`	`3`

Hint 1: height(t) = 1 + max(height(left), height(right)); null has height 0.

Exercise 2 · Connected components — *Intermediate*

Read n, m and m undirected edges. Print the number of connected components.

Example input	Expected output
`6 3 ⏎ 0 1 ⏎ 1 2 ⏎ 3 4`	`3`

Hint 1: Start a BFS from every unvisited vertex and count the starts.

Exercise 3 · Shortest path length — *Challenge*

Read n, m and the edges of an undirected graph. Print the minimum number of edges from vertex 0 to vertex n−1, or -1.

Example input	Expected output
5 5 ⏎ 0 1 ⏎ 0 2 ⏎ 1 3 ⏎ 2 3 ⏎ 3 4	3

Hint 1: BFS with a distance array.

4.3 Searching: Linear and Binary Search

Exercise 1 · First occurrence — *Beginner*

Read n, target and n numbers (unsorted). Print the index of the first occurrence of the target, or -1.

Example input	Expected output
5 7 ⏎ 4 9 7 1 7	2

Hint 1: A for loop with an early exit.

Exercise 2 · Binary search — *Intermediate*

Read n, target and a SORTED array of distinct numbers. Print the index of the target or -1 using binary search.

Example input	Expected output
6 9 ⏎ 1 3 5 7 9 11	4

Hint 1: lo, hi, mid; discard half each time.

Exercise 3 · Count occurrences — *Challenge*

In a sorted array (duplicates allowed) print how many times the target occurs, in O(log n).

Example input	Expected output
8 2 ⏎ 1 2 2 2 2 3 5 9	4

Hint 1: lowerBound(target + 1) − lowerBound(target).

4.4 Elementary Sorting: Bubble, Insertion, Selection; Stability

Exercise 1 · Bubble sort with swap count — *Beginner*

Read n numbers. Bubble-sort them; print the sorted array and, on the next line, the number of swaps performed.

Example input	Expected output
5 ⏎ 4 3 2 1 5	1 2 3 4 5 ⏎ 6

Hint 1: The swap count equals the number of inversions.

Exercise 2 · Insertion steps — *Intermediate*

Read n numbers. After each insertion step i = 1..n−1 print the whole array.

Example input	Expected output
4 ⏎ 5 2 4 1	2 5 4 1 ⏎ 2 4 5 1 ⏎ 1 2 4 5

Hint 1: Print after placing the key.

Exercise 3 · Stable by key — *Challenge*

Read n pairs `key label`. Sort them by key with a stable algorithm and print the labels in order, separated by spaces.

Example input	Expected output
5 ⏎ 2 a ⏎ 1 b ⏎ 2 c ⏎ 1 d ⏎ 2 e	b d a c e

Hint 1: Insertion sort with a strict > comparison on the keys.

4.5 Merge Sort and Divide and Conquer

Exercise 1 · Merge sort — *Beginner*

Read n numbers and print them sorted using your own merge sort.

Example input	Expected output
7 ⏎ 38 27 43 3 9 82 10	3 9 10 27 38 43 82

Hint 1: Recurse on halves, then merge with a temp array.

Exercise 2 · Merge two sorted arrays — *Intermediate*

Read n, m and two sorted arrays. Print their merge as one sorted array.

Example input	Expected output
3 4 ⏎ 1 4 9 ⏎ 2 3 10 11	1 2 3 4 9 10 11

Hint 1: Two pointers; take the smaller front value.

Exercise 3 · Count inversions — *Challenge*

Read n (≤ 10^5) numbers and print the number of pairs i < j with a[i] > a[j]. Use O(n log n).

Example input	Expected output
5 ⏎ 2 4 1 3 5	3

Hint 1: During the merge, when you take from the right half add mid − i.

4.6 Iteration versus Recursion

Exercise 1 · gcd both ways — *Beginner*

Read a and b. Print the gcd computed iteratively and recursively, separated by a space.

Example input	Expected output
84 36	12 12

Hint 1: The loop and the recursion use the same rule.

Exercise 2 · Climbing stairs — *Intermediate*

You may climb 1 or 2 steps at a time. Read n and print the number of distinct ways to climb n steps, using a loop.

Example input	Expected output
10	89

Hint 1: ways(n) = ways(n−1) + ways(n−2): Fibonacci.

Exercise 3 · Reverse a string — *Beginner*

Read a word. Print its reverse computed recursively, a space, then computed with a loop.

Example input	Expected output
stressed	desserts desserts

Hint 1: reverse(s) = reverse(s.substring(1)) + s.charAt(0).

4.7 Time Complexity and Big-O Notation

Exercise 1 · Count the triangle — *Beginner*

Read n and print how many times the innermost statement runs in: for i = 1..n, for j = 1..i, count++.

Example input	Expected output
5	15

Hint 1: 1 + 2 + … + n = n(n+1)/2.

Exercise 2 · Passes of a halving loop — *Beginner*

Read n and print how many times `n /= 2` runs in `while (n > 1)`.

Example input	Expected output
1000	9

Hint 1: $\lfloor \log_2 n \rfloor$.

Exercise 3 · The crossover point — *Challenge*

An O(n²) algorithm does n² steps; an O(n log₂ n) one does `factor · n · log₂ n` steps. Read factor and print the smallest n ≥ 2 for which the quadratic algorithm does MORE steps.

Example input	Expected output
10	59

Hint 1: Search upwards from n = 2.

4.8 Algorithm Design Paradigms

Exercise 1 · Activity selection (greedy) — *Intermediate*

Read n intervals `start end`. Print the maximum number of mutually non-overlapping intervals (touching endpoints are allowed).

Example input	Expected output
5 ⏎ 1 3 ⏎ 2 4 ⏎ 3 5 ⏎ 0 6 ⏎ 5 7	3

Hint 1: Sort by end time; always take the earliest-finishing compatible interval.

Exercise 2 · Fewest coins (DP) — *Intermediate*

Read k, the k coin values and an amount. Print the fewest coins needed, or -1.

Example input	Expected output
3 ⏎ 4 3 1 ⏎ 6	2

Hint 1: best[x] = 1 + min over coins of best[x − c].

Exercise 3 · No two adjacent ones (backtracking) — *Challenge*

Read n (≤ 10). Print all binary strings of length n with no two consecutive 1s, in increasing order.

Example input	Expected output
3	000 ⏎ 001 ⏎ 010 ⏎ 100 ⏎ 101

Hint 1: Recurse by length; only append 1 if the last char is not 1.

4.9 Analysing Loops: Counting Iterations

Exercise 1 · Tripling outer loop — *Intermediate*

Read n. Count how many times `count++` runs in: for (i = 1; i <= n; i *= 3) for (j = 0; j < n; j++) count++.

Example input	Expected output
30	120

Hint 1: The outer loop runs $\lfloor \log_3 n \rfloor + 1$ times.

Exercise 2 · Half-linear times log — *Intermediate*

Read n. Count: for (i = n/2; i >= 1; i--) for (j = 1; j <= n; j *= 2) count++.

Example input	Expected output
16	40

Hint 1: (n/2) outer × ($\lfloor \log_2 n \rfloor + 1$) inner.

Exercise 3 · Measure the growth ratio — *Challenge*

For the triangular loop (j ≤ i) let count(n) be the iteration count. Read n and print count(2n) / count(n) with one decimal.

Example input	Expected output
500	4.0

Hint 1: A ratio near 4 means quadratic growth.

4.10 Analysing Recursion: Recurrence Relations

Exercise 1 · Count the calls — *Intermediate*

Read n. Count the calls of f where f(n) = f(n−1) + f(n−1) for n > 1 and returns 1 for n ≤ 1 (count every call including the first).

Example input	Expected output
10	1023

Hint 1: calls(n) = 1 + 2·calls(n−1), calls(1) = 1 → $2^n - 1$.

Exercise 2 · Two half calls — *Intermediate*

g(n) = g(n/2) + g(n/2) for n > 1 (integer division), 0 for n ≤ 1. Count all calls including the first for the input n.

Example input	Expected output
16	31

Hint 1: For n a power of two the tree has 2n − 1 nodes.

Exercise 3 · Karatsuba work — *Challenge*

Define t(n): if n ≤ 1 add 1 to a counter; otherwise add n, then call t(n/2) three times. For n a power of two print the final counter value.

Example input	Expected output
8	65

Hint 1: Level k has 3^k nodes each adding n/2^k; leaves add 1.

4.11 Space Complexity and the Call Stack

Exercise 1 · Maximum recursion depth — *Beginner*

f(n) = f(n−1) + 1 for n > 1, f(1) = 1. Read n and print the maximum number of simultaneously active calls.

Example input	Expected output
50	50

Hint 1: It equals n.

Exercise 2 · Fibonacci in O(1) space — *Intermediate*

Read n (≤ 90) and print the n-th Fibonacci number (F(0) = 0, F(1) = 1) using only two variables.

Example input	Expected output
50	12586269025

Hint 1: Keep the last two values and slide them forward.

Exercise 3 · Duplicates with O(1) extra space — *Challenge*

Read n numbers; print YES if any value appears twice. Do it by sorting the array in place (no HashSet).

Example input	Expected output
5 ⏎ 4 1 7 1 9	YES

Hint 1: After sorting, duplicates are adjacent.

4.12 Best, Average and Worst Case

Exercise 1 · Comparisons in a linear search — *Beginner*

Read n, a target and n numbers. Print how many comparisons a linear search makes (n if the target is absent).

Example input	Expected output
5 7 ⏎ 4 9 7 1 7	3

Hint 1: Stop counting at the first match.

Exercise 2 · Average successful search — *Beginner*

Read n. Print the average number of comparisons of a successful linear search over n elements when every position is equally likely, with one decimal.

Example input	Expected output
9	5.0

Hint 1: (1 + 2 + … + n) / n = (n + 1) / 2.

Exercise 3 · Insertion sort comparisons — *Intermediate*

Read n numbers and print the number of comparisons insertion sort makes (compare the key with elements to its left until one is ≤ key).

Example input	Expected output
5 ⏎ 5 4 3 2 1	10

Hint 1: A reversed array of 5 gives 4+3+2+1.

Part 5 · Archive Algorithm Mastery

5.1 Euclid's Algorithm (GCD)

Exercise 1 · Reduce a fraction — *Beginner*

Read a numerator and a denominator (both positive). Print the fraction in lowest terms as `numerator/denominator`.

- 1 ≤ numerator, denominator ≤ 10^9

Example input	Expected output
12 18	2/3

Hint 1: What number can you divide BOTH parts by without leaving a remainder? **Hint 2:** Use Euclid's algorithm for that number, then divide.

Exercise 2 · LCM of a list — *Intermediate*

Read n and then n positive integers. Print their least common multiple. The answer fits in a `long`.

- 1 ≤ n ≤ 20
- 1 ≤ each number ≤ 10^6
- answer ≤ $9 \cdot 10^{18}$

Example input	Expected output
3 ⏎ 4 6 8	24

Hint 1: lcm(a, b) = a / gcd(a, b) * b. **Hint 2:** Fold the list: lcm = lcm(lcm, next). **Hint 3:** Use long and divide before multiplying.

Exercise 3 · Modular inverse — *Challenge*

Given a and m, print the number x in [0, m−1] such that (a · x) mod m = 1, or -1 if no such x exists.

- 1 ≤ a, m ≤ 10^9
- m ≥ 2

Example input	Expected output
3 11	4
6 9	−1

Hint 1: An inverse exists exactly when gcd(a, m) = 1. **Hint 2:** Extended Euclid finds x and y with a·x + m·y = gcd(a, m). Then x is the inverse. **Hint 3:** The x it returns can be negative — shift it into [0, m−1] with ((x % m) + m) % m.

5.2 Euler's Phi (Totient) Function

Exercise 1 · List the coprime numbers — *Beginner*

Read n. Print, in increasing order and separated by spaces, every k with 1 ≤ k ≤ n and gcd(k, n) = 1.

- 1 ≤ n ≤ 10^5

Example input	Expected output
12	1 5 7 11

Hint 1: You need a gcd helper — reuse Euclid. **Hint 2:** Build the output in a StringBuilder and print once.

Exercise 2 · Sum of totients — *Intermediate*

Read n and print φ(1) + φ(2) + ... + φ(n).

- $1 \le n \le 10^6$

Example input	Expected output
10	32

Hint 1: Computing φ separately for each i with trial division is too slow in the worst case — use the table approach. **Hint 2:** Remember the answer can exceed int range for large n: use long for the sum.

Exercise 3 · Coprime pairs — *Challenge*

Count ordered pairs (a, b) with 1 ≤ a, b ≤ n and gcd(a, b) = 1.

- $1 \le n \le 10^6$

Example input	Expected output
3	7

Hint 1: For a fixed larger value b ≥ 2, how many a < b are coprime to b? **Hint 2:** Each such unordered pair counts twice (a,b) and (b,a). Don't forget (1,1).

5.3 Chinese Remainder Theorem

Exercise 1 · Two congruences — *Beginner*

Given coprime moduli a and b and remainders r1, r2 (read in the order `a r1 b r2`), print the smallest non-negative x with x % a = r1 and x % b = r2.

- 2 ≤ a, b ≤ 1000, gcd(a, b) = 1
- 0 ≤ r1 < a, 0 ≤ r2 < b

Example input	Expected output
3 2 5 3	8

Hint 1: The answer is below a·b. **Hint 2:** Loop x from 0 and test both conditions.

Exercise 2 · CRT with the formula — *Intermediate*

Read n, then n lines `m r`. All moduli are pairwise coprime. Print the smallest non-negative x with x ≡ r (mod m) for all lines.

- 1 ≤ n ≤ 5
- 2 ≤ m ≤ 1000
- product of moduli ≤ 10^{12}

Example input	Expected output
3 ⏎ 3 2 ⏎ 5 3 ⏎ 7 2	23

Hint 1: M_i = M / m_i. **Hint 2:** Inverse of M_i modulo m_i: extended Euclid, or try 1..m_i−1. **Hint 3:** Reduce mod M after every multiplication.

Exercise 3 · Non-coprime moduli — *Challenge*

Read `r1 m1 r2 m2` (the moduli may share factors). Print the smallest non-negative x with x ≡ r1 (mod m1) and x ≡ r2 (mod m2), or -1 if none exists.

- 1 ≤ m1, m2 ≤ 10^6
- 0 ≤ r1 < m1, 0 ≤ r2 < m2

Example input	Expected output
2 6 4 10	14
2 6 1 4	−1

Hint 1: Only numbers r1, r1+m1, r1+2·m1, … satisfy the first congruence. **Hint 2:** Their remainders modulo m2 repeat after at most m2 steps — try that many.

5.4 Combinations (nCr)

Exercise 1 · nCr as a number — *Beginner*

Read n and r (0 ≤ r ≤ n ≤ 30). Print C(n, r).

Example input	Expected output
5 2	10

Hint 1: Multiply first, then divide, one step at a time. **Hint 2:** Use long.

Exercise 2 · nCr modulo 1 000 000 007 — *Intermediate*

Read n and r (0 ≤ r ≤ n ≤ 1000). Print C(n, r) mod 1 000 000 007.

Example input	Expected output
10 3	120

Hint 1: The true value is far too large for long — use Pascal's rule and take % MOD after each addition.

Exercise 3 · List all combinations — *Challenge*

Read n and r (1 ≤ r ≤ n ≤ 10). Print every r-element subset of {1..n}, one per line, elements in increasing order, subsets in lexicographic order.

Example input	Expected output
4 2	1 2 ⏎ 1 3 ⏎ 1 4 ⏎ 2 3 ⏎ 2 4 ⏎ 3 4

Hint 1: Recursion: choose(start, count). **Hint 2:** Only pick values larger than the previous one.

5.5 Karatsuba's Multiplication

Exercise 1 · School-book multiplication — *Beginner*

Read two non-negative integers as digit strings (up to 100 digits each). Print their product without using BigInteger or `long` arithmetic on the whole numbers.

Example input	Expected output
12345 6789	83810205

Hint 1: Digit i of x times digit j of y contributes to position i + j + 1 of the result. **Hint 2:** Keep a carry as you add.

Exercise 2 · Count the base multiplications — *Intermediate*

Run the lesson's Karatsuba algorithm (split at n/2 low digits; base case when either factor < 10) on x and y (both < 10^9). Print how many times the base case is reached.

Example input	Expected output
1234 5678	11

Hint 1: Add a static counter incremented in the base case. **Hint 2:** Do not change the splitting rule — the answer depends on it.

Exercise 3 · Big Karatsuba — *Challenge*

Read two non-negative integers (up to 1000 digits). Multiply them with your own Karatsuba on BigInteger (do not call BigInteger.multiply except below a 30-bit threshold) and print the product.

Example input	Expected output
123456789 987654321	121932631112635269

Hint 1: Use bitLength() to decide the split point. **Hint 2:** Use shiftLeft/shiftRight and subtract to obtain the low part.

5.6 Booth's Algorithm (signed binary multiplication)

Exercise 1 · n-bit two's complement — *Beginner*

Read an integer x and a width n (1 ≤ n ≤ 16, x fits in n bits). Print the n-bit two's complement bit pattern of x.

Example input	Expected output
−5 8	11111011

Hint 1: Walk from the top bit n−1 down to 0 and print (x >> bit) & 1. **Hint 2:** Java's >> on a negative int already yields the two's complement bits.

Exercise 2 · Booth's operations — *Intermediate*

Read an n-bit multiplier as a bit string. For each cycle (from the rightmost bit to the leftmost), print the operation Booth's algorithm performs: `−M`, `+M` or `0`. Separate with spaces.

Example input	Expected output
0110	0 −M 0 +M

Hint 1: Keep the previous bit (starting at 0). **Hint 2:** (1,0) → −M; (0,1) → +M; otherwise 0.

Exercise 3 · Booth multiplication — *Challenge*

Read two signed integers (each fits in 16-bit two's complement). Print their product, computed by simulating Booth's algorithm on registers (no `*`).

Example input	Expected output
−3 4	−12

Hint 1: Use width = 16 (17 if the multiplicand is −32768). **Hint 2:** Mask AC and M to the register width after every addition. **Hint 3:** Combine AC and Q at the end and sign-extend the 2n-bit value.

5.7 Simple Sieve of Eratosthenes

Exercise 1 · Count primes — *Beginner*

Read n and print how many primes are ≤ n.

- $1 \le n \le 10^7$

Example input	Expected output
100	25

Hint 1: Use a boolean array of size n + 1. **Hint 2:** Count while you sieve.

Exercise 2 · The k-th prime — *Intermediate*

Read k ($1 \le k \le 100\,000$) and print the k-th prime (2 is the 1st).

Example input	Expected output
1000	7919

Hint 1: How large can the 100 000-th prime be? About 1.3 million. **Hint 2:** Sieve up to a fixed safe bound and count.

Exercise 3 · Prime factorisation queries — *Challenge*

Read q, then q numbers x ($2 \le x \le 10^6$). For each, print its prime factorisation as `x = p1^e1 * p2^e2` (exponent omitted when 1).

Example input	Expected output
3 ⏎ 12 ⏎ 97 ⏎ 360	12 = 2^2 * 3 ⏎ 97 = 97 ⏎ 360 = 2^3 * 3^2 * 5

Hint 1: Build an array spf[x] = smallest prime factor with a sieve. **Hint 2:** Repeatedly divide x by spf[x].

5.8 Segmented & Incremental Sieve

Exercise 1 · Primes in a small range — *Beginner*

Read L and R ($1 \le L \le R \le 10^6$). Print how many primes are in [L, R].

Example input	Expected output
10 30	6

Hint 1: A plain sieve up to R is fine here. **Hint 2:** Count only values ≥ max(L, 2).

Exercise 2 · Primes in a huge range — *Intermediate*

Read L and R with $R \le 10^{12}$ and $R - L \le 10^6$. Print how many primes lie in [L, R].

Example input	Expected output
1000000000000 1000000000100	4

Hint 1: The simple sieve cannot allocate 10^{12} cells. **Hint 2:** Base primes up to 10^6, then window marking.

Exercise 3 · Largest prime gap in a window — *Challenge*

Read L and R ($R - L \le 10^6$, $R \le 10^{12}$). Print the largest difference between two consecutive primes in [L, R], or -1 if there are fewer than two primes.

Example input	Expected output
1 100	8

Hint 1: Sieve the window, then scan keeping the previous prime.

5.9 Binary Palindrome

Exercise 1 · Is it a binary palindrome? — *Beginner*

Read a non-negative integer n. Print YES if its binary representation is a palindrome, otherwise NO.

- $0 \le n \le 2^{31} - 1$

Example input	Expected output
5	YES

Example input	Expected output
10	NO

Hint 1: Compare the number with its bit reversal. **Hint 2:** Remember 0 is written "0" — a palindrome.

Exercise 2 · Count binary palindromes up to n — *Intermediate*

Read n. Print how many integers from 1 to n have a palindromic binary representation.

- $1 \le n \le 10^6$

Example input	Expected output
10	5

Hint 1: Loop and reuse your checker. **Hint 2:** Check your answer by listing them for n = 10.

Exercise 3 · Next binary palindrome — *Challenge*

Read n. Print the smallest binary palindrome strictly greater than n.

- $0 \le n \le 10^9$

Example input	Expected output
7	9

Hint 1: Palindromes are fairly dense: the gap is never larger than about 2^(bits/2). **Hint 2:** Start at n + 1 and keep going.

5.10 Swap Two Nibbles in a Byte

Exercise 1 · Swap the nibbles — *Beginner*

Read an integer 0 ≤ x ≤ 255 and print the value obtained by swapping its two 4-bit halves.

Example input	Expected output
171	186

Hint 1: High nibble: (x & 0xF0) >>> 4 **Hint 2:** Low nibble: (x & 0x0F) << 4

Exercise 2 · Swap adjacent bits — *Intermediate*

Read an unsigned 32-bit integer and swap bit 0 with bit 1, bit 2 with bit 3, and so on. Print the result as an unsigned number.

Example input	Expected output
23	43

Hint 1: Even-position bits: x & 0x55555555; odd-position bits: x & 0xAAAAAAAA. **Hint 2:** Shift evens left by 1 and odds right by 1 (use >>>). **Hint 3:** Print `result & 0xFFFFFFFFL` to see it unsigned.

Exercise 3 · Reverse 32 bits — *Challenge*

Read an unsigned 32-bit integer and print the integer whose binary form is the original's 32-bit pattern reversed.

Example input	Expected output
43261596	964176192

Hint 1: Loop 32 times: shift result left, OR in the lowest bit of x, shift x right with >>>. **Hint 2:** Print as unsigned (long).

5.11 Longest Sequence of 1s (and after flipping a bit)

Exercise 1 · Longest run of 1s — *Beginner*

Read an int and print the length of the longest block of consecutive 1 bits in its 32-bit two's-complement representation.

Example input	Expected output
56825	6

Hint 1: Scan 32 bits with `x & 1` and `x >>>= 1`.

Exercise 2 · Flip one bit — *Intermediate*

Read an int. You may flip exactly one bit (0 → 1 or leave it). Print the longest run of 1s you can obtain among the 32 bits.

Example input	Expected output
478	8

Hint 1: Brute force is fine: try each of the 32 bits. **Hint 2:** Or track current and previous runs in one pass.

Exercise 3 · Flip up to k zeros — *Challenge*

Read an int x and k. Print the longest run of 1s in the 32-bit representation after flipping at most k zeros to ones.

- $0 \le k \le 32$

Example input	Expected output
478 2	9

Hint 1: Sliding window over the 32 bits: keep the number of zeros inside the window ≤ k. **Hint 2:** Move the left end forward whenever the window has too many zeros.

5.12 Josephus Problem

Exercise 1 · Elimination order — *Beginner*

Read N and k. Print the numbers in the order in which people are removed; the survivor is printed last.

Example input	Expected output
5 2	2 4 1 5 3

Hint 1: Keep an ArrayList 1..N and an index. **Hint 2:** index = (index + k − 1) % size, then remove(index).

Exercise 2 · Survivor for large N — *Intermediate*

Read N and k and print the survivor's position (1-based).

- $1 \le N \le 10^7$
- $1 \le k \le 10^9$

Example input	Expected output
10 3	4

Hint 1: Simulation is too slow for $N = 10^7$ — use the recurrence. **Hint 2:** Use long for (ans + k).

Exercise 3 · Josephus with k = 2 and N up to 10^{18} — *Challenge*

Every second person is removed (k = 2). Read N and print the survivor.

- $1 \le N \le 10^{18}$

Example input	Expected output
100	73

Hint 1: Write N = 2^m + r. What happens after r eliminations? **Hint 2:** Long.highestOneBit gives 2^m.

5.13 Block Swap Algorithm (Array Rotation)

Exercise 1 · Left rotation — *Beginner*

Read n, d and an array of n integers. Print the array after rotating it left by d positions.

- $1 \le n \le 10^5$
- $0 \le d \le 10^9$

Example input	Expected output
5 2 ⏎ 1 2 3 4 5	3 4 5 1 2

Hint 1: Only d mod n matters. **Hint 2:** Output element i is input element (i + d) mod n.

Exercise 2 · Right rotation in place — *Intermediate*

Read n, d and the array. Print the array rotated RIGHT by d, using O(1) extra memory (reversals).

Example input	Expected output
5 2 ⏎ 1 2 3 4 5	4 5 1 2 3

Hint 1: A right rotation by d is a left rotation by n − d. **Hint 2:** Reverse three times.

Exercise 3 · How many rotations? — *Challenge*

A sorted array of distinct integers was rotated left an unknown number of times. Read n and the array and print how many left rotations produced it (the index of the minimum). Use O(log n) time.

Example input	Expected output
6 ⏎ 15 18 2 3 6 12	2

Hint 1: The minimum is the only element smaller than its predecessor. **Hint 2:** Binary search: compare a[mid] with a[hi].

5.14 Leaders in an Array

Exercise 1 · Print the leaders — *Beginner*

Read n and an array. Print all leaders (strictly greater than every element to their right) from left to right.

Example input	Expected output
6 ⏎ 16 17 4 3 5 2	17 5 2

Hint 1: Scan from the right with a running maximum. **Hint 2:** Store, then print in reverse.

Exercise 2 · Non-strict leaders — *Intermediate*

Same as before, but an element counts as a leader when it is greater than OR EQUAL to every element on its right.

Example input	Expected output
5 ⏎ 7 10 4 10 6	10 10 6

Hint 1: Only the comparison changes.

Exercise 3 · Next greater element — *Challenge*

For each element print the first element to its right that is strictly greater, or -1 if there is none.

Example input	Expected output
4 ⏎ 4 5 2 25	5 25 25 −1

Hint 1: Scan from the right with a stack of candidates. **Hint 2:** Pop candidates that are not greater than the current element.

5.15 Majority Element (Boyer–Moore Voting)

Exercise 1 · Majority with a HashMap — *Beginner*

Read n and an array. Print the majority element or the text `No majority element`.

Example input	Expected output
5 ⏎ 2 2 1 1 1	1

Hint 1: Count frequencies with a HashMap<Integer,Integer>. **Hint 2:** Majority means more than n/2.

Exercise 2 · Boyer–Moore — *Intermediate*

Same task, but use O(1) extra memory: no sorting, no hash map.

- $1 \le n \le 10^6$

Example input	Expected output
7 ⏎ 3 3 4 2 4 4 2	No majority element

Hint 1: Two variables: candidate and count. **Hint 2:** Verify with a second pass.

Exercise 3 · Elements appearing more than n/3 times — *Challenge*

Print all values that occur more than $\lfloor n/3 \rfloor$ times, in increasing order, or `None`.

Example input	Expected output
8 ⏎ 1 1 1 3 3 2 2 2	1 2

Hint 1: At most two values can exceed n/3. **Hint 2:** Keep TWO candidates with two counters; decrement both when a third value arrives. **Hint 3:** Verify both candidates in a second pass.

5.16 Maximum Product Subarray

Exercise 1 · Count subarrays with positive product — *Beginner*

Read n and an array. Print the number of subarrays whose product is strictly positive.

- $1 \le n \le 2000$

Example input	Expected output
5 ⏎ 2 3 −2 4 0	4

Hint 1: A subarray is positive when it has no zero and an even number of negatives. **Hint 2:** For each start extend until a zero appears.

Exercise 2 · Maximum product subarray — *Intermediate*

Read n and an array of integers. Print the largest product of a contiguous non-empty subarray.

- $1 \le n \le 10^5$
- $-10 \le a[i] \le 10$
- the answer fits in a 64-bit signed integer

Example input	Expected output
4 ⏎ 2 3 −2 4	6

Hint 1: Keep both max and min ending at i. **Hint 2:** Swap them when a[i] is negative.

Exercise 3 · Which subarray? — *Challenge*

Print three numbers: the maximum product, then the start and end index (0-based) of the first subarray that achieves it (earliest start, then earliest end).

- $1 \le n \le 1000$

Example input	Expected output
4 ⏎ 2 3 −2 4	6 0 1

Hint 1: The brute force $O(n^2)$ is fine for $n \le 1000$. **Hint 2:** Only update on a STRICTLY larger product to keep the first one.

5.17 Maximum Equilibrium Sum

Exercise 1 · First equilibrium index — *Beginner*

Read n and an array. Print the smallest equilibrium index (0-based) or -1.

Example input	Expected output
7 ⏎ −7 1 5 2 −4 3 0	3

Hint 1: right = total − left − a[i]. **Hint 2:** Stop at the first match.

Exercise 2 · Maximum equilibrium sum — *Intermediate*

Print the largest side-sum over all equilibrium indices (left sum excluding a[i]), or the word `none`.

Example input	Expected output
9 ⏎ 1 2 3 4 5 4 3 2 1	10

Hint 1: Collect a maximum over the equilibria. **Hint 2:** Use a boolean for "found" instead of a sentinel.

Exercise 3 · Balanced split points — *Challenge*

Count the split positions k ($1 \le k \le n-1$) such that the sum of a[0..k−1] equals the sum of a[k..n−1].

- $2 \le n \le 10^5$

Example input	Expected output
6 ⏎ 1 −1 1 −1 1 −1	2

Hint 1: Both parts together sum to total, so the prefix must equal total/2. **Hint 2:** One pass with a running prefix.

5.18 Maximum Sum of an Hourglass in a Matrix

Exercise 1 · One hourglass — *Beginner*

Read a 3×3 matrix (nine integers) and print its hourglass sum.

Example input	Expected output
`1 1 1 ⏎ 0 1 0 ⏎ 1 1 1`	7

Hint 1: Skip m[1][0] and m[1][2].

Exercise 2 · Maximum hourglass with negatives — *Intermediate*

Read R, C and the matrix (R, C ≥ 3). Print the maximum hourglass sum.

- 3 ≤ R, C ≤ 100
- −100 ≤ value ≤ 100

Example input	Expected output
`3 3 ⏎ −1 −1 −1 ⏎ −1 −9 −1 ⏎ −1 −1 −1`	−15

Hint 1: Start the maximum at Integer.MIN_VALUE.

Exercise 3 · Largest k×k block — *Challenge*

Read R, C, k and a matrix. Print the maximum sum of any k×k sub-matrix in O(R·C) time using 2-D prefix sums.

- 1 ≤ k ≤ min(R, C) ≤ 1000

Example input	Expected output
`3 3 2 ⏎ 1 2 3 ⏎ 4 5 6 ⏎ 7 8 9`	28

Hint 1: p[i][j] = sum of the rectangle (0,0)…(i−1,j−1). **Hint 2:** Block sum = p[i][j] − p[i−k][j] − p[i][j−k] + p[i−k][j−k].

5.19 Toggle the Switch (100 Doors)

Exercise 1 · Count the open doors — *Beginner*

Read n (≤ 1000) and simulate the monkeys. Print the number of doors that are open at the end.

Example input	Expected output
`10`	3

Hint 1: boolean[] and two loops.

Exercise 2 · A trillion doors — *Intermediate*

Read n up to 10^{18}. Print how many doors are open.

Example input	Expected output
`1000000000000000000`	1000000000

Hint 1: Open doors are perfect squares. **Hint 2:** Use an exact integer square root.

Exercise 3 · Only the first m monkeys — *Challenge*

Only monkeys 1..m are released (m ≤ n). Read n and m and print how many doors are open. (n ≤ 10^6)

Example input	Expected output
10 3	4

Hint 1: A door is toggled once per divisor ≤ m. **Hint 2:** Simulation over m monkeys is O(n log m).

5.20 Alice's Apple Tree (Minimum Garden Perimeter)

Exercise 1 · Apples in a square — *Beginner*

Read r. Print the number of apples inside the square with corners (±r, ±r).

Example input	Expected output
3	168

Hint 1: Use the closed form 2r(r+1)(2r+1). **Hint 2:** Check r = 1: 12.

Exercise 2 · Minimum perimeter (small) — *Intermediate*

Read neededApples (≤ 10^9) and print the minimum perimeter.

Example input	Expected output
13	16

Hint 1: Add the rings $12r^2$ until the total is ≥ needed.

Exercise 3 · Minimum perimeter (large) — *Challenge*

Read neededApples (≤ 10^{15}) and print the minimum perimeter, using binary search.

Example input	Expected output
1000000000000000	503968

Hint 1: apples(r) = 2r(r+1)(2r+1) is increasing. **Hint 2:** hi = 200000 is a valid upper bound.

5.21 Weighted Substring (Smallest String of a Given Weight)

Exercise 1 · Weight of a string — *Beginner*

Read a string of uppercase letters and print its total weight (A=1, B=3, C=12, ..., with weight(n) = (n+1)·weight(n−1)).

- length ≤ 10
- only letters A–J

Example input	Expected output
ACC	25

Hint 1: Build the weight table once. **Hint 2:** Sum table[c − 'A'].

Exercise 2 · Smallest string — *Intermediate*

Read a weight W (1 ≤ W ≤ 10^{16}) and print the shortest string of weight W, choosing the lexicographically smallest among the shortest.

Example input	Expected output
25	ACC

Example input	Expected output
20	AABBC

Hint 1: Greedy from the heaviest letter that fits. **Hint 2:** Sort at the end.

Exercise 3 · Normalise a string — *Challenge*

Read a string of uppercase letters (up to 10^5 characters). Print the shortest, lexicographically smallest string with the same total weight — without ever computing the (possibly huge) total.

Example input	Expected output
AAAAB	ABB

Hint 1: (i + 2) copies of letter i weigh the same as one copy of letter i + 1 (A: 3 copies = B; B: 4 copies = C). **Hint 2:** Count letters, then carry from A upwards like long addition.

5.22 Move Hyphen to the Beginning

Exercise 1 · First hyphen to the front — *Beginner*

Read a line. If it contains a hyphen, move its FIRST hyphen to the beginning; otherwise print the line unchanged.

Example input	Expected output
face-prep	-faceprep

Hint 1: indexOf, then substring twice.

Exercise 2 · All hyphens to the front — *Intermediate*

Move every hyphen to the beginning of the line, keeping the other characters in order.

- length ≤ 10^6

Example input	Expected output
a-b-c	--abc

Hint 1: Two StringBuilders, or one char array scanned from the right.

Exercise 3 · Minimum adjacent swaps — *Challenge*

Print the minimum number of swaps of ADJACENT characters needed to bring all hyphens to the front.

- length ≤ 10^6 (answer may need long)

Example input	Expected output
a-b-c	3

Hint 1: Each hyphen must pass every non-hyphen character that stands before it. **Hint 2:** Sum, over hyphens, the number of non-hyphen characters seen so far.

5.23 Strobogrammatic Number

Exercise 1 · Is it strobogrammatic? — *Beginner*

Read a number as a string. Print YES if it looks the same rotated by 180°, else NO.

Example input	Expected output
69	YES
25	NO

Hint 1: Two pointers from both ends. **Hint 2:** Allowed rotations: 0-0, 1-1, 8-8, 6-9, 9-6.

Exercise 2 · How many of length n? — *Intermediate*

Read n (1 ≤ n ≤ 18). Print how many n-digit strobogrammatic numbers have no leading zero (for n = 1 the digit 0 counts).

Example input	Expected output
4	20

Hint 1: The outermost pair has 4 choices (11, 69, 88, 96). **Hint 2:** Each inner pair has 5 choices; an odd middle digit has 3.

Exercise 3 · Strobogrammatic in a range — *Challenge*

Read low and high (as strings of up to 15 digits, low ≤ high). Print how many strobogrammatic numbers lie in [low, high].

Example input	Expected output
50 100	3

Hint 1: Generate by length from len(low) to len(high). **Hint 2:** For equal lengths, string comparison equals numeric comparison.

5.24 Lexicographically First Palindromic Rearrangement

Exercise 1 · Can it be a palindrome? — *Beginner*

Read a word (letters only, case-insensitive). Print YES if its letters can be rearranged into a palindrome, else NO.

Example input	Expected output
aabb	YES

Hint 1: Count letters; how many have an odd count?

Exercise 2 · Build the smallest palindrome — *Intermediate*

Read a word. Print the lexicographically first palindrome using ALL its letters (case-insensitive, print in lowercase), or `no palindromic string`.

Example input	Expected output
Malayalam	aalmymlaa
apple	no palindromic string

Hint 1: TreeMap<Character,Integer> keeps letters sorted. **Hint 2:** front = each letter repeated count/2 times.

Exercise 3 · How many palindromes? — *Challenge*

Read a word of up to 20 letters. Print the number of DISTINCT palindromes that can be formed from all its letters (0 if impossible).

Example input	Expected output
aabbcc	6

Hint 1: Only the left half can be chosen freely. **Hint 2:** Number of arrangements of a multiset: m! / (c_1! c_2! ...) where counts are halved.

5.25 Manacher's Algorithm (Longest Palindromic Substring)

Exercise 1 · Is it a palindrome? — *Beginner*

Read a word. Print YES if it is a palindrome, else NO.

Example input	Expected output
racecar	YES

Hint 1: Two pointers moving inwards.

Exercise 2 · Longest palindromic substring (expand) — *Intermediate*

Read a word (length ≤ 2000). Print its longest palindromic substring; if several have the same length, print the one that starts first.

Example input	Expected output
cbbd	bb

Hint 1: Two kinds of centres. **Hint 2:** Update only when strictly longer to keep the earliest.

Exercise 3 · Count palindromic substrings — *Challenge*

Read a word of up to 10^5 letters. Print how many substrings (counted by position) are palindromes, in O(n).

Example input	Expected output
abba	6

Hint 1: In the padded string a radius r around a centre contributes ⌈r/2⌉ palindromes. **Hint 2:** Use long.

5.26 Natural Sort Order

Exercise 1 · Sort file names naturally — *Beginner*

Read n and then n file names. Print them in natural order, one per line (case-insensitive text, numbers compared numerically).

Example input	Expected output
4 ⏎ img12.png ⏎ img10.png ⏎ img2.png ⏎ IMG1.png	IMG1.png ⏎ img2.png ⏎ img10.png ⏎ img12.png

Hint 1: Chunk, compare chunk by chunk. **Hint 2:** List.sort(comparator).

Exercise 2 · Sort version numbers — *Intermediate*

Read n and n version strings like 1.2.10. Print them in increasing version order on one line (a missing component counts as 0).

Example input	Expected output
4 ⏎ 1.10 ⏎ 1.2.10 ⏎ 1.2.9 ⏎ 1.9	1.2.9 1.2.10 1.9 1.10

Hint 1: split("\\.") and compare component by component as numbers.

Exercise 3 · Natural order with deterministic ties — *Challenge*

Sort names naturally, ignoring case in text chunks. If two names are numerically equal chunk by chunk (e.g. file01 and file1), the SHORTER name comes first, then plain compareTo. Print on one line separated by spaces.

Example input	Expected output
4 ⏎ file01 ⏎ file1 ⏎ File2 ⏎ file10	file1 file01 File2 file10

Hint 1: Compare chunk by chunk, then the number of chunks, then total length, then compareTo.

5.27 Maneuvering (Counting Grid Paths)

Exercise 1 · Paths in a rectangle — *Beginner*

Read m and n and print the number of right/down paths from the top-left to the bottom-right of an m × n grid.

- 1 ≤ m, n ≤ 18

Example input	Expected output
3 7	28

Hint 1: dp table with first row and column set to 1.

Exercise 2 · Paths with obstacles — *Intermediate*

Read R, C and a grid of 0 (free) and 1 (blocked). Print the number of right/down paths from (0,0) to (R−1,C−1) that avoid blocked cells (0 if the start or end is blocked).

- 1 ≤ R, C ≤ 100
- the answer fits in a long

Example input	Expected output
3 3 ⏎ 0 0 0 ⏎ 0 1 0 ⏎ 0 0 0	2

Hint 1: A blocked cell contributes 0. **Hint 2:** dp[0][0] = 1 only if the start is free.

Exercise 3 · Huge grid modulo a prime — *Challenge*

Read m and n (≤ 100 000). Print the number of paths modulo 1 000 000 007.

Example input	Expected output
3 7	28

Hint 1: A DP table of 10^{10} cells is impossible — use C(m+n−2, m−1). **Hint 2:** Precompute factorials mod p and use a modular inverse (Fermat: a^(p−2)).

5.28 Sorted Distinct Permutations

Exercise 1 · How many distinct permutations? — *Beginner*

Read a word (≤ 20 letters). Print the number of distinct permutations.

Example input	Expected output
AAB	3

Hint 1: n! divided by the factorial of every letter count. **Hint 2:** long is enough for n ≤ 20.

Exercise 2 · Print them sorted — *Intermediate*

Read a word (≤ 8 letters). Print every distinct permutation, one per line, in alphabetical order.

Example input	Expected output
ABA	AAB ⏎ ABA ⏎ BAA

Hint 1: Counts array over the 256 characters. **Hint 2:** Try letters in increasing order.

Exercise 3 · The k-th permutation — *Challenge*

Read a word (≤ 15 letters) and k. Print the k-th distinct permutation in sorted order (1-based) WITHOUT generating the others, or `NONE` if k is too large.

Example input	Expected output
DBCA 10	BCDA

Hint 1: At each position, for each letter, count how many permutations start with it: arrangements of the remaining multiset. **Hint 2:** If k ≤ that count, take the letter; otherwise subtract and continue.

5.29 N-Queens (Backtracking)

Exercise 1 · Validate a placement — *Beginner*

Read n and then n integers: the column (1-based) of the queen in each row. Print VALID if no two queens attack each other, else INVALID.

Example input	Expected output
4 ⏎ 2 4 1 3	VALID

Hint 1: Same column, or |Δrow| == |Δcol|, means attack.

Exercise 2 · Count solutions — *Intermediate*

Read N (1 ≤ N ≤ 10) and print the number of distinct ways to place N non-attacking queens.

Example input	Expected output
6	4

Hint 1: Row by row; three boolean arrays for column and diagonals.

Exercise 3 · Lexicographically first solution — *Challenge*

Read N (1 ≤ N ≤ 12). Print the solution that is smallest when read as the column of the queen in row 1, row 2, … (columns 1-based, separated by spaces), or NONE.

Example input	Expected output
8	1 5 8 6 3 7 2 4

Hint 1: Trying columns in increasing order and stopping at the first success gives the lexicographically first.

5.30 Maze Solving (Rat in a Maze)

Exercise 1 · How many cells can the rat reach? — *Beginner*

Read R, C and an R × C maze (0 road, 1 wall, 2 entry, 3 exit). Print the number of non-wall cells reachable from the entry by moving up, down, left or right (the entry itself counts).

Example input	Expected output
3 3 ⏎ 2 0 1 ⏎ 1 0 1 ⏎ 1 0 3	5

Hint 1: Flood fill with recursion or a stack. **Hint 2:** Mark cells visited.

Exercise 2 · Is there a way out? — *Intermediate*

Same input. Print YES if the exit can be reached from the entry, else NO.

Example input	Expected output
3 3 ⏎ 2 1 3 ⏎ 0 1 0 ⏎ 0 0 0	YES

Hint 1: DFS returning boolean.

Exercise 3 · Shortest path — *Challenge*

Same input. Print the minimum number of moves from the entry to the exit, or -1.

- 1 ≤ R, C ≤ 500

Example input	Expected output
3 3 ⏎ 2 0 0 ⏎ 1 1 0 ⏎ 3 0 0	6

Hint 1: Breadth-first search with a queue and a distance array.

5.31 Hamiltonian Cycle

Exercise 1 · Verify a cycle — *Beginner*

Read V, the V × V adjacency matrix, and then V+1 vertex numbers. Print YES if they form a Hamiltonian cycle (start = end, every vertex once, consecutive vertices adjacent).

Example input	Expected output
4 ⏎ 0 1 0 1 ⏎ 1 0 1 0 ⏎ 0 1 0 1 ⏎ 1 0 1 0 ⏎ 0 1 2 3 0	YES

Hint 1: Check first = last, no repeats among the first V, and an edge between consecutive vertices.

Exercise 2 · Find a cycle — *Intermediate*

Read V and the matrix. Print `YES` followed by the first Hamiltonian cycle found starting at 0 (trying vertices in increasing order) — e.g. `YES 0 1 2 4 3 0` — or `NO`.

- 3 ≤ V ≤ 10

Example input	Expected output
5 ⏎ 0 1 0 1 0 ⏎ 1 0 1 1 1 ⏎ 0 1 0 0 1 ⏎ 1 1 0 0 0 ⏎ 0 1 1 0 0	NO

Hint 1: Backtracking with path[] and the closing check.

Exercise 3 · Count Hamiltonian cycles — *Challenge*

Read V (≤ 10) and the matrix. Count the Hamiltonian cycles that start and end at vertex 0, counting a cycle and its reverse as two different cycles.

Example input	Expected output
4 ⏎ 0 1 1 1 ⏎ 1 0 1 1 ⏎ 1 1 0 1 ⏎ 1 1 1 0	6

Hint 1: Same search but do not stop at the first success. **Hint 2:** K_4 has 3! = 6 orderings of the other vertices.

5.32 Knight's Tour & Warnsdorff's Rule

Exercise 1 · Verify a tour — *Beginner*

Read n and an n × n board of move numbers 1..n². Print YES if every number appears once and consecutive numbers are a knight's move apart, else NO.

Example input	Expected output
5 ⏎ 1 6 15 10 21 ⏎ 14 9 20 5 16 ⏎ 19 2 7 22 11 ⏎ 8 13 24 17 4 ⏎ 25 18 3 12 23	YES

Hint 1: Store the position of each number. **Hint 2:** A knight move has |dx|, |dy| = (1, 2) or (2, 1).

Exercise 2 · Degree of a square — *Intermediate*

Read n, x, y (0-based) and print how many squares an unrestricted knight on (x, y) can move to on an n × n board.

Example input	Expected output
8 0 0	2

Hint 1: Test all eight moves against the board limits.

Exercise 3 · Reachable squares — *Challenge*

Read n and k. A knight starts at (0, 0) on an n × n board. Print how many squares it can occupy after EXACTLY k moves (it may revisit squares).

Example input	Expected output
8 2	10

Hint 1: Keep a boolean grid of "possible positions" and advance it k times.

5.33 Selection Sort

Exercise 1 · Sort and count swaps — *Beginner*

Read n and an array. Sort it with selection sort and print the sorted array on one line, then the number of swaps that actually moved something (skip a swap when minIndex = i).

Example input	Expected output
5 ⏎ 64 25 12 22 11	11 12 22 25 64 ⏎ 3

Hint 1: Count a swap only when minIndex != i.

Exercise 2 · Largest first — *Intermediate*

Sort an array in DESCENDING order using the same idea (select the maximum). Print on one line.

Example input	Expected output
5 ⏎ 3 1 4 1 5	5 4 3 1 1

Hint 1: Only the comparison changes.

Exercise 3 · K smallest, cheaply — *Challenge*

Read n, k and an array. Print the k smallest values in increasing order using only k passes of selection (O(n·k)).

- 1 ≤ k ≤ n ≤ 5000

Example input	Expected output
6 3 ⏎ 7 10 4 3 20 15	3 4 7

Hint 1: You do not have to finish the sort: stop after k passes.

5.34 Quick Sort

Exercise 1 · Count the partitions — *Beginner*

Quick sort the array with the lesson's Lomuto code. Read n and the array; print the sorted array. (Implement quick sort yourself; the output is simply the sorted array.)

Example input	Expected output
7 ⏎ 8 3 1 7 0 10 2	0 1 2 3 7 8 10

Hint 1: partition returns the pivot's final index. **Hint 2:** Recurse on both sides.

Exercise 2 · Descending quick sort — *Intermediate*

Sort the array in descending order using quick sort.

Example input	Expected output
5 ⏎ 3 1 4 1 5	5 4 3 1 1

Hint 1: Flip the comparison in the partition: move elements ≥ pivot to the left region.

Exercise 3 · k-th smallest with quickselect — *Challenge*

Read n, k and an array. Print the k-th smallest element (1-based) in expected O(n) time without fully sorting.

Example input	Expected output
6 3 ⏎ 7 10 4 3 20 15	7

Hint 1: After a partition the pivot is at index p. If p == k−1 you are done; otherwise only recurse into ONE side.